

CS 342: Computer Vision and Pattern Recognition

Instructor: Br. Tamal Mj

Shanti Mantra

Lyrics in Sanskrit

Meaning in English

ॐ सह नाववतु।

Om, May we all be protected

सह नौ भुनक्तु।

May we all be nourished

सह वीर्यं करवावहै।

May we work together with great energy

तेजस्यि नावधीतमस्तु

May our intellect be sharpened (may our study be effective)

मा विद्विषावहै।

Let there be no Animosity amongst us

ॐ शान्तिः शान्तिः शान्तिः ॥

Om, peace (in me), peace (in nature), peace (in divine forces)

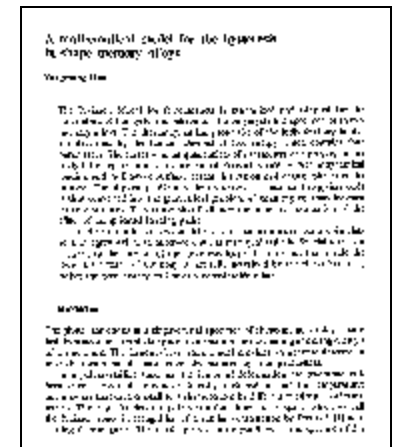
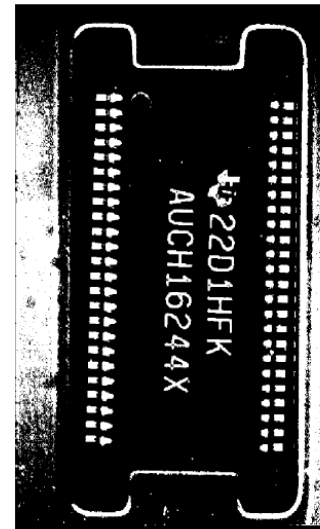
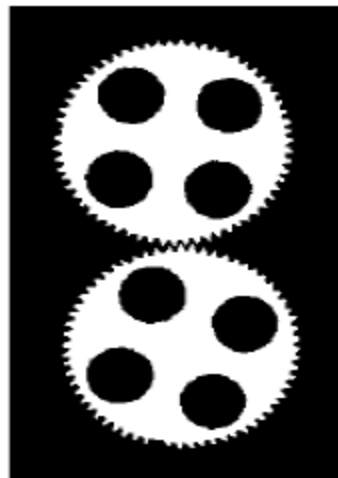
Today

- Binary image analysis
- Readings:
- Digital Image Processing (DIP):
 - Chapter 9: Morphological Image Processing
 - Especially: Pg 635-648
 - Chapter 10 Section 10.3: Thresholding
 - Pg: 742-764

Binary images



0	0	0	0	0
1	1	1	1	1
2	2	2	2	2
3	3	3	3	3
4	4	4	4	4
5	5	5	5	5
6	6	6	6	6
7	7	7	7	7
8	8	8	8	8
9	9	9	9	9



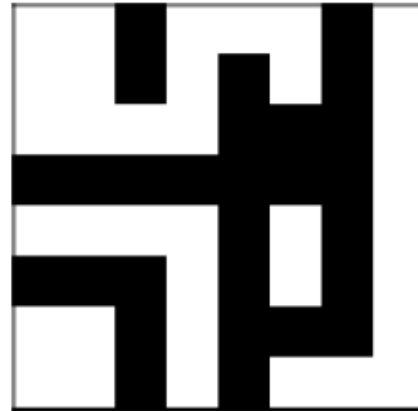
Binary image analysis: basic steps

- Convert the image into binary form
 - Thresholding
- Clean up the thresholded image
 - Morphological operators

Binary images

- Two pixel values
 - Foreground and background
 - Mark region(s) of interest

1	1	0	1	1	1	0	1
1	1	0	1	0	1	0	1
1	1	1	1	0	0	0	1
0	0	0	0	0	0	0	1
1	1	1	1	0	1	0	1
0	0	0	1	0	1	0	1
1	1	0	1	0	0	0	1
1	1	0	1	0	1	1	1



Thresholding

- Given a grayscale image or an intermediate matrix $\rightarrow$ threshold to create a binary output.

Example: edge detection



Gradient magnitude



```
fg_pix = find(gradient_mag > t);
```

Looking for pixels where gradient is strong.

Thresholding

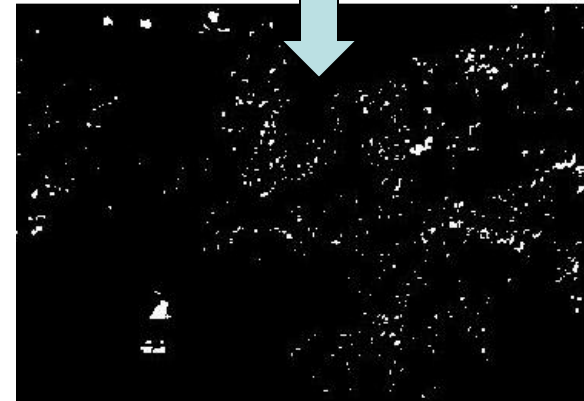
- Given a grayscale image or an intermediate matrix $\rightarrow$ threshold to create a binary output.

Example: background subtraction



Looking for pixels that differ significantly from the “empty” background.

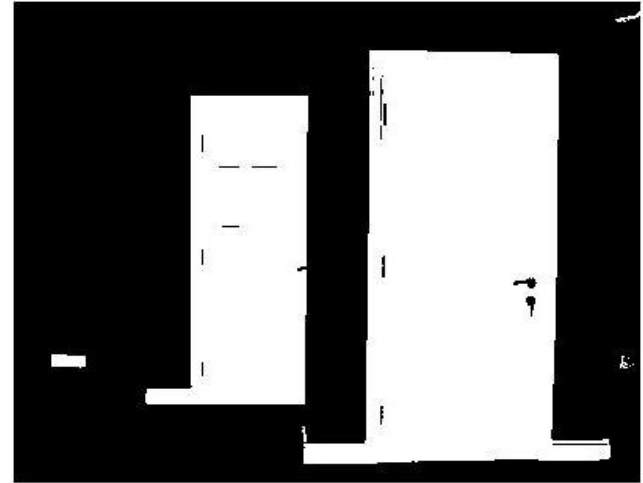
```
fg_pix = find(diff > t);
```



Thresholding

- Given a grayscale image or an intermediate matrix → threshold to create a binary output.

Example: intensity-based detection



```
fg_pix = find(im < 65);
```

Looking for dark pixels

Thresholding

- Given a grayscale image or an intermediate matrix → threshold to create a binary output.

Example: color-based detection



```
fg_pix = find(hue > t1 & hue < t2);
```

Looking for pixels within a certain hue range.

Adaptive Thresholding

- Variance-Based Thresholding
- Otsu's Method
- Please read:
 - Davies: 4.5.1 Variance-Based Thresholding
 - Tutorial: Adaptive Thresholding (Pg. 53-58)
 - Wikipedia:
https://en.wikipedia.org/wiki/Otsu%27s_method

How Adaptive Thresholding Works

1. **Divide the Image into Regions:** The image is split into small regions or neighborhoods.
2. **Calculate Local Thresholds:** For each region, a threshold value is computed based on statistical properties such as:
 - **Mean:** The average intensity of the local region.
 - **Gaussian-weighted Mean:** A weighted mean where closer pixels contribute more.
 - **Median:** The middle intensity value of the local region.
3. **Apply Thresholding:** Each pixel is compared with the calculated local threshold and assigned a binary value:
 - If $I(x, y) > T(x, y)$, set pixel to white (255).
 - Otherwise, set it to black (0).

Methods of Adaptive Thresholding

- **Mean Adaptive Thresholding:** Uses the mean intensity of the neighborhood as the threshold. [local]
- **Gaussian Adaptive Thresholding:** Uses a weighted sum of neighboring pixels (Gaussian kernel) to determine the threshold. [local]
- **Otsu's Method (Hybrid Approach):** Finds an optimal global threshold dynamically
 - but can be applied locally in a block-wise manner too.

Otsu's Method

- A good many investigators have studied this sort of
- Among the most wellknown approaches are the variance-based methods.
- In these methods, the image intensity histogram is analyzed to find where it can best be partitioned to optimize criteria based on ratios of the within-class, between-class, and total variance.

Otsu's Method

- calculate the between-class variance

First, we assume that the image has a grayscale resolution of L gray levels. The number of pixels with gray level i is written as n_i , so the total number of pixels in the image is $N = n_1 + n_2 + \dots + n_L$. Thus, the probability of a pixel having gray level i is:

$$p_i = \frac{n_i}{N} \quad (4.4)$$

where

$$p_i \geq 0 \quad \sum_{i=1}^L p_i = 1 \quad (4.5)$$

Otsu's Method

For ranges of intensities up to and above the threshold value k , we can now calculate the between-class variance σ_B^2 and the total variance σ_T^2 :

$$\sigma_B^2 = \pi_0(\mu_0 - \mu_T)^2 + \pi_1(\mu_1 - \mu_T)^2 \quad (4.6)$$

$$\sigma_T^2 = \sum_{i=1}^L (i - \mu_T)^2 p_i \quad (4.7)$$

where

$$\pi_0 = \sum_{i=1}^k p_i \quad \pi_1 = \sum_{i=k+1}^L p_i = 1 - \pi_0 \quad (4.8)$$

$$\mu_0 = \sum_{i=1}^k ip_i / \pi_0 \quad \mu_1 = \sum_{i=k+1}^L ip_i / \pi_1 \quad \mu_T = \sum_{i=1}^L ip_i \quad (4.9)$$

Making use of the latter definitions, the formula for the between-class variance can be simplified to:

$$\sigma_B^2 = \pi_0 \pi_1 (\mu_1 - \mu_0)^2 \quad (4.10)$$

Otsu's Method

Between-class Variance:

$$\sigma_B^2 = \pi_0 \pi_1 (\mu_1 - \mu_0)^2$$

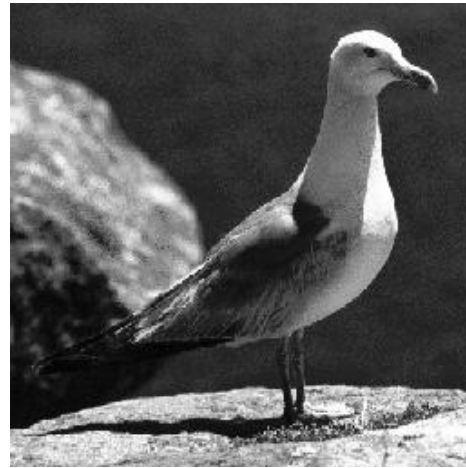
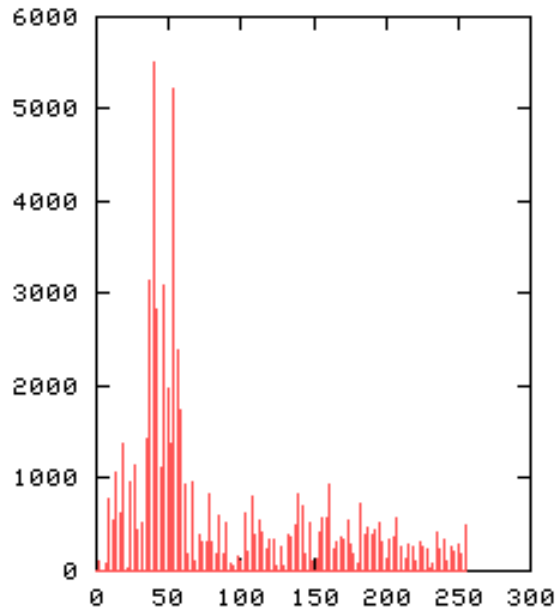
For a single threshold, the criterion to be maximized is the ratio of the between-class variance to the total variance:

$$\eta = \frac{\sigma_B^2}{\sigma_T^2} \quad (4.11)$$

However, the total variance is constant for a given image histogram, so maximizing η simplifies to maximizing the between-class variance.

The method can readily be extended to the dual threshold case $1 \leq k_1 \leq k_2 \leq L$, where the resultant classes, C_0 , C_1 , and C_2 , have respective gray-level ranges of $[1, \dots, k_1]$, $[k_1 + 1, \dots, k_2]$, and $[k_2 + 1, \dots, L]$.

Image Representations: Histograms

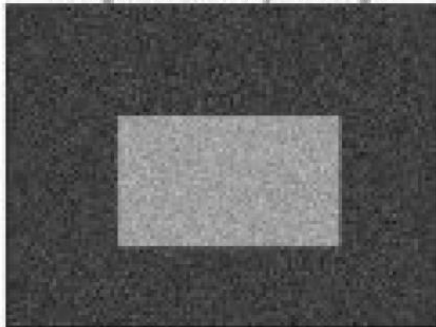


Global histogram

- Represent distribution of features
 - Color, texture, depth, ...

Otsu's Method

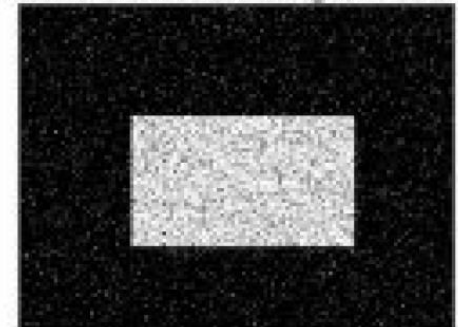
Original Noisy Image



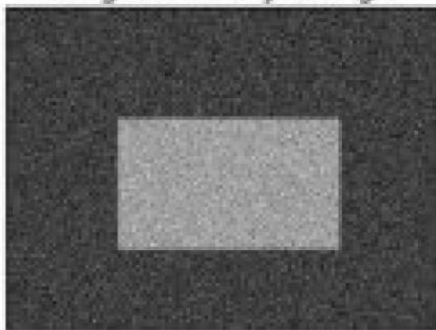
Histogram



Global Thresholding ($v=127$)



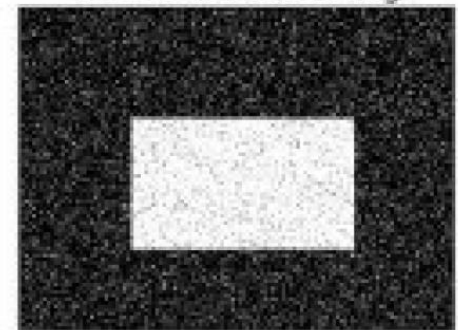
Original Noisy Image



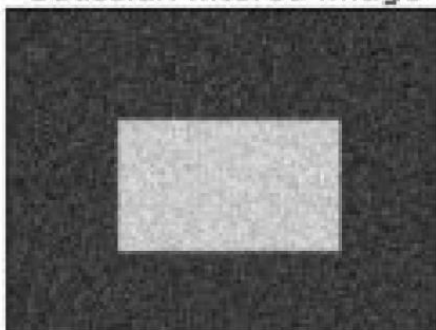
Histogram



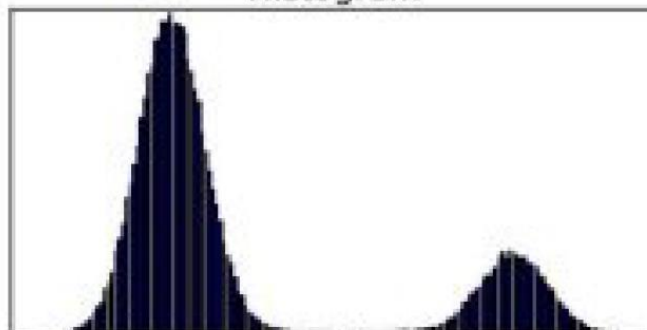
Otsu's Thresholding



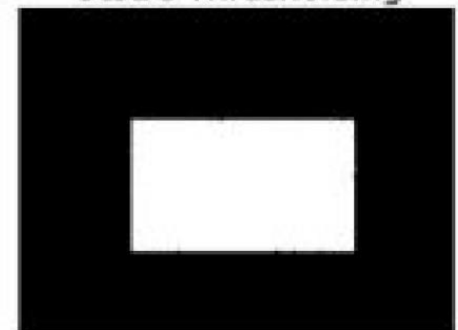
Gaussian filtered Image



Histogram

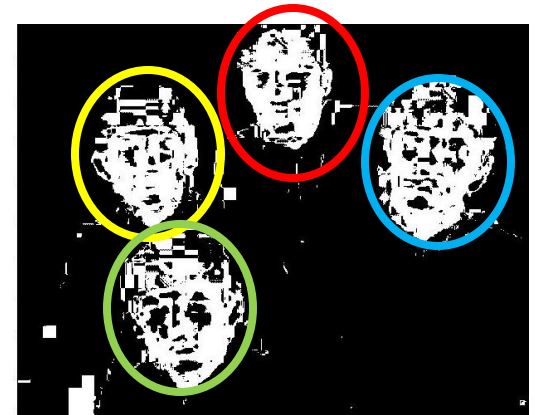
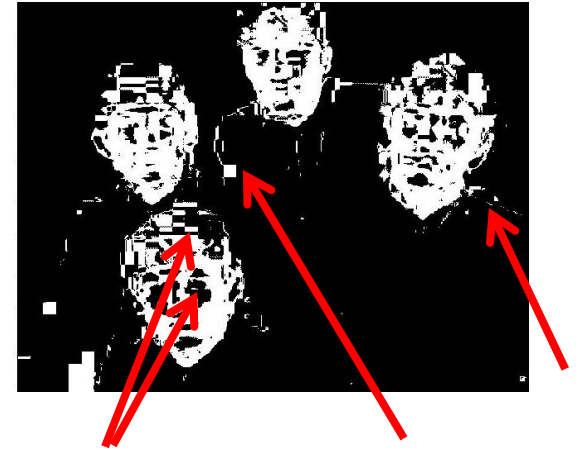


Otsu's Thresholding



Issues

- What to do with “noisy” binary outputs?
 - Holes
 - Extra small fragments
- How to demarcate multiple regions of interest?
 - Count objects
 - Compute further features per object

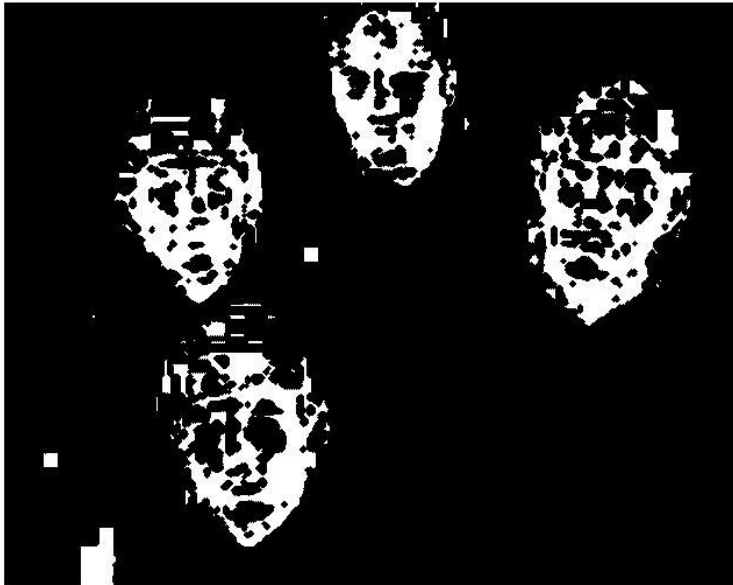


Morphological operators

- Change the shape of the foreground regions via intersection/union operations between a scanning structuring element and binary image.
- Useful to clean up result from thresholding
- Basic operators are:
 - Dilation
 - Erosion

Dilation

- Expands connected components
- Grow features
- Fill holes

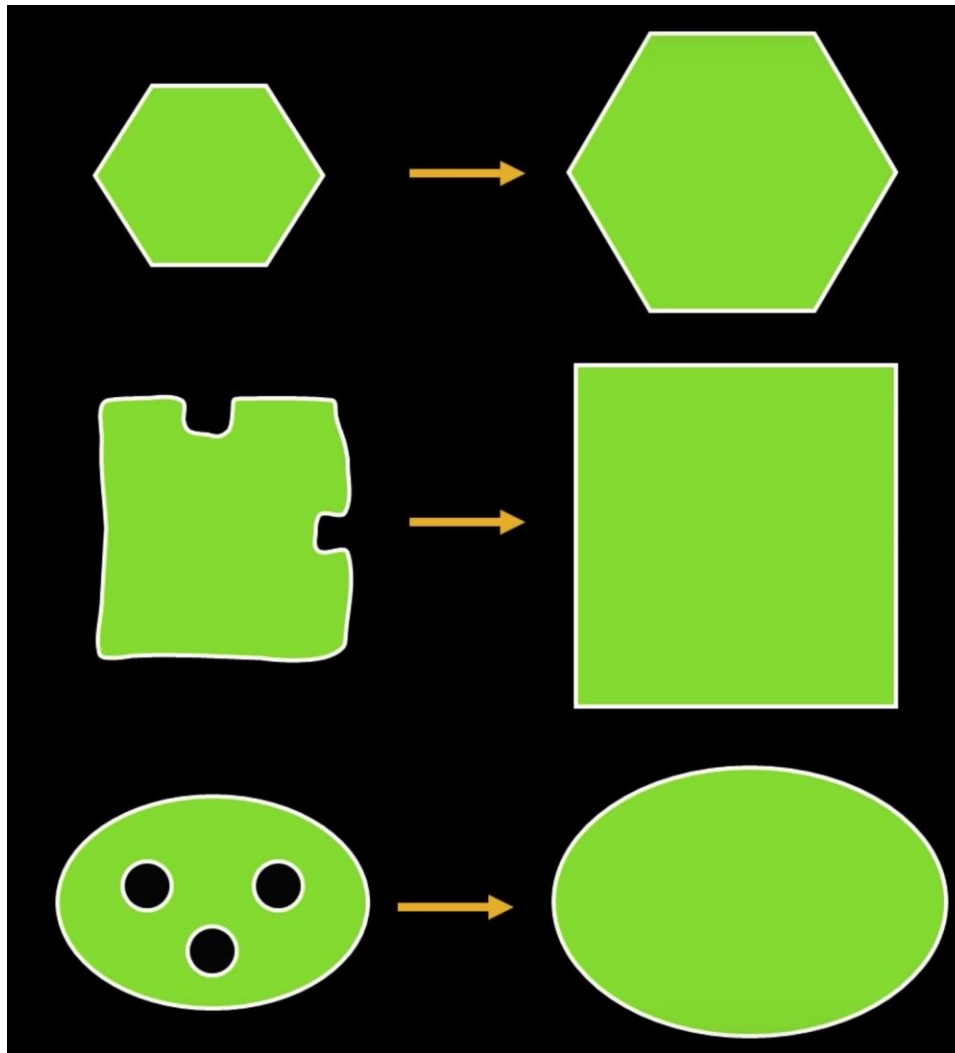


Before dilation



After dilation

Dilation

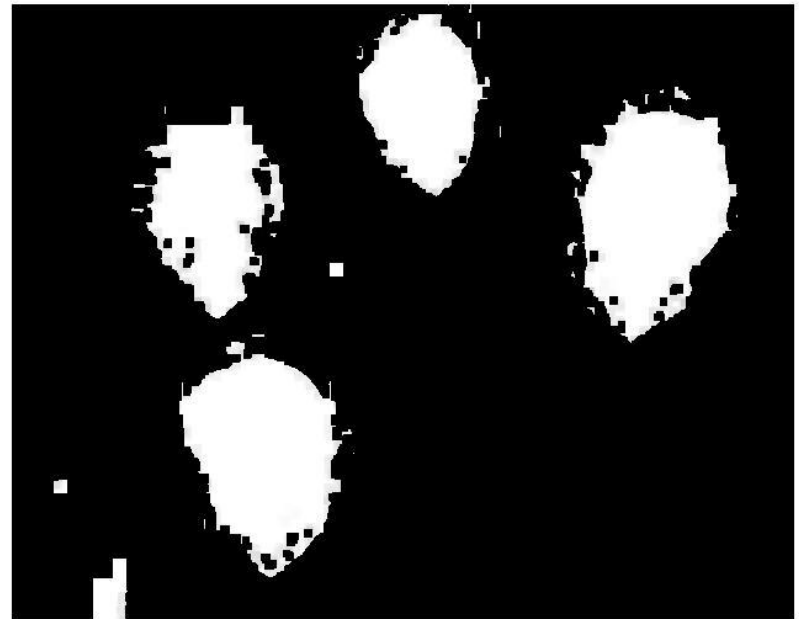


Erosion

- Erode connected components
- Shrink features
- Remove bridges, branches, noise

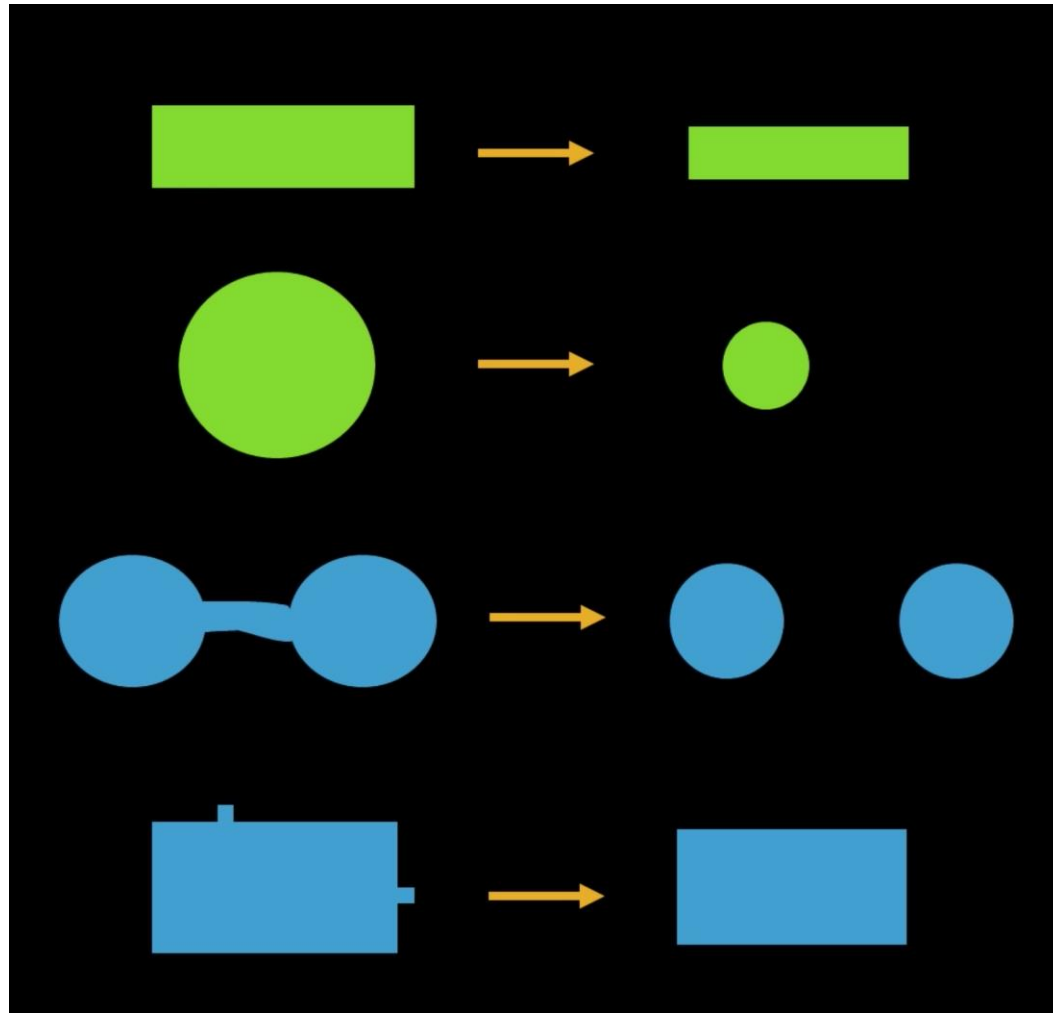


Before erosion



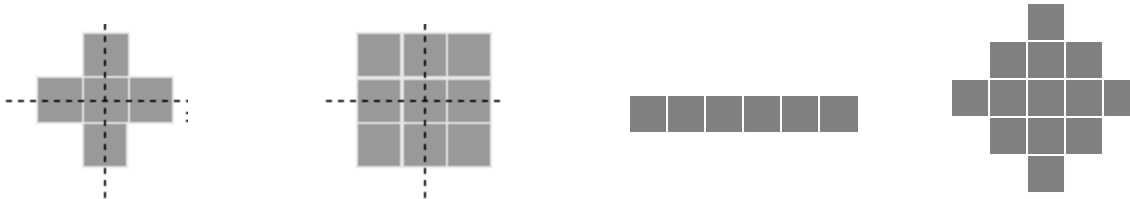
After erosion

Erosion



Structuring elements

- **Masks** of varying shapes and sizes used to perform morphology, for example:



- Scan mask across foreground pixels to transform the binary image

Dilation vs. Erosion

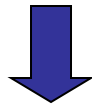
At each position:

- **Dilation:** if current pixel is foreground, OR the structuring element with the input image.

Example for Dilation (1D)

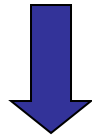
Input image

1	0	0	0	1	1	1	0	1	1
---	---	---	---	---	---	---	---	---	---



1	1	1
---	---	---

Structuring Element



Output Image

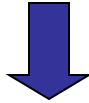
1									
---	--	--	--	--	--	--	--	--	--

$$g(x) = f(x) \oplus SE$$

Example for Dilation

Input image

1	0	0	0	1	1	1	0	1	1
---	---	---	---	---	---	---	---	---	---



Structuring Element

1	1	1
---	---	---

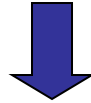
Output Image

1	1								
---	---	--	--	--	--	--	--	--	--

Example for Dilation

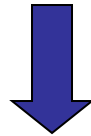
Input image

1	0	0	0	1	1	1	0	1	1
---	---	---	---	---	---	---	---	---	---



Structuring Element

1	1	1
---	---	---



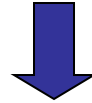
Output Image

1	1	0							
---	---	---	--	--	--	--	--	--	--

Example for Dilation

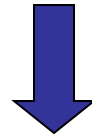
Input image

1	0	0	0	1	1	1	0	1	1
---	---	---	---	---	---	---	---	---	---



Structuring Element

1	1	1
---	---	---



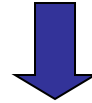
Output Image

1	1	0	1						
---	---	---	---	--	--	--	--	--	--

Example for Dilation

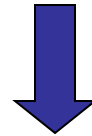
Input image

1	0	0	0	1	1	1	0	1	1
---	---	---	---	---	---	---	---	---	---



Structuring Element

1	1	1
---	---	---



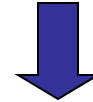
Output Image

1	1	0	1	1	1				
---	---	---	---	---	---	--	--	--	--

Example for Dilation

Input image

1	0	0	0	1	1	1	0	1	1
---	---	---	---	---	---	---	---	---	---



Structuring Element

1	1	1
---	---	---



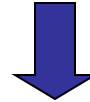
Output Image

1	1	0	1	1	1	1			
---	---	---	---	---	---	---	--	--	--

Example for Dilation

Input image

1	0	0	0	1	1	1	0	1	1
---	---	---	---	---	---	---	---	---	---



Structuring Element

1	1	1
---	---	---



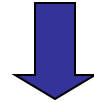
Output Image

1	1	0	1	1	1	1	1		
---	---	---	---	---	---	---	---	--	--

Example for Dilation

Input image

1	0	0	0	1	1	1	0	1	1
---	---	---	---	---	---	---	---	---	---



Structuring Element

1	1	1
---	---	---

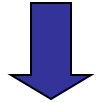
Output Image

1	1	0	1	1	1	1	1		
---	---	---	---	---	---	---	---	--	--

Example for Dilation

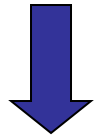
Input image

1	0	0	0	1	1	1	0	1	1
---	---	---	---	---	---	---	---	---	---



Structuring Element

1	1	1
---	---	---



Output Image

1	1	0	1	1	1	1	1	1	1
---	---	---	---	---	---	---	---	---	---

Note that the object gets bigger and holes are filled.

2D example for dilation

1	1	1	1	1	1	1	
			1	1	1	1	
			1	1	1	1	
		1	1	1	1	1	
			1	1	1	1	
		1	1				

(a) Binary image **B**

1	1	1
1	1	1
1	1	1

(b) Structuring element **S**

1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1
	1	1	1	1	1	1	1
	1	1	1	1	1	1	1
	1	1	1	1	1	1	1
	1	1	1	1	1	1	1
	1	1	1	1			

(c) Dilation $\mathbf{B} \oplus \mathbf{S}$

Dilation vs. Erosion

Dilation: A pixel is set to foreground if **at least one** pixel under the structuring element in the input image is foreground.

Erosion: A pixel is set to foreground if **all** pixels under the structuring element's nonzero entries are also foreground in the input image.

Dilation vs. Erosion

Dilation: A pixel in the output is **foreground (1)** if **at least one** of the pixels covered by the structuring element in the input image is foreground.

- This means that if any pixel under the structuring element (SE) corresponds to a foreground pixel, the output pixel will also be foreground.
- **More formally:**

$$A \oplus B = \{z \mid (B_z \cap A) \neq \emptyset\}$$

Erosion: A pixel in the output is **foreground (1)** only if **all** the pixels under the structuring element's nonzero positions are also foreground.

- This means the structuring element must be completely contained in the foreground region for the output pixel to be foreground.
- **More formally:**

$$A \ominus B = \{z \mid B_z \subseteq A\}$$

A: The input binary image (the foreground object you are processing).

B: The structuring element (SE),

Example for Erosion (1D)

Input image

1	0	0	0	1	1	1	0	1	1
---	---	---	---	---	---	---	---	---	---



1	1	1
---	---	---

Structuring Element



Output Image

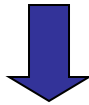
0									
---	--	--	--	--	--	--	--	--	--

$$g(x) = f(x) \ominus SE$$

Example for Erosion (1D)

Input image

1	0	0	0	1	1	1	0	1	1
---	---	---	---	---	---	---	---	---	---



Structuring Element

1	1	1
---	---	---



Output Image

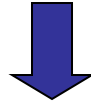
0	0								
---	---	--	--	--	--	--	--	--	--

$$g(x) = f(x) \ominus SE$$

Example for Erosion

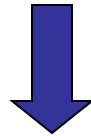
Input image

1	0	0	0	1	1	1	0	1	1
---	---	---	---	---	---	---	---	---	---



Structuring Element

1	1	1
---	---	---



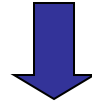
Output Image

0	0	0							
---	---	---	--	--	--	--	--	--	--

Example for Erosion

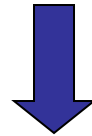
Input image

1	0	0	0	1	1	1	0	1	1
---	---	---	---	---	---	---	---	---	---



Structuring Element

1	1	1
---	---	---



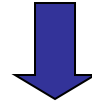
Output Image

0	0	0	0						
---	---	---	---	--	--	--	--	--	--

Example for Erosion

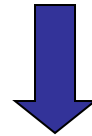
Input image

1	0	0	0	1	1	1	0	1	1
---	---	---	---	---	---	---	---	---	---



Structuring Element

1	1	1
---	---	---



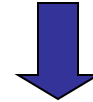
Output Image

0	0	0	0	0					
---	---	---	---	---	--	--	--	--	--

Example for Erosion

Input image

1	0	0	0	1	1	1	0	1	1
---	---	---	---	---	---	---	---	---	---



Structuring Element

1	1	1
---	---	---



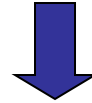
Output Image

0	0	0	0	0	1				
---	---	---	---	---	---	--	--	--	--

Example for Erosion

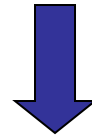
Input image

1	0	0	0	1	1	1	0	1	1
---	---	---	---	---	---	---	---	---	---



Structuring Element

1	1	1
---	---	---



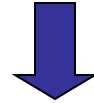
Output Image

0	0	0	0	0	1	0			
---	---	---	---	---	---	---	--	--	--

Example for Erosion

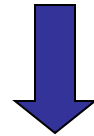
Input image

1	0	0	0	1	1	1	0	1	1
---	---	---	---	---	---	---	---	---	---



Structuring Element

1	1	1
---	---	---



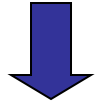
Output Image

0	0	0	0	0	1	0	0		
---	---	---	---	---	---	---	---	--	--

Example for Erosion

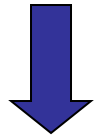
Input image

1	0	0	0	1	1	1	0	1	1
---	---	---	---	---	---	---	---	---	---



Structuring Element

1	1	1
---	---	---



Output Image

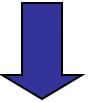
0	0	0	0	0	1	0	0	0	
---	---	---	---	---	---	---	---	---	--

Example for Erosion

Input image

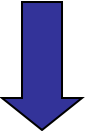
1	0	0	0	1	1	1	0	1	1
---	---	---	---	---	---	---	---	---	---

Structuring Element



1	1	1
---	---	---

Output Image



0	0	0	0	0	1	0	0	0	1
---	---	---	---	---	---	---	---	---	---

Note that the object gets smaller

2D example for erosion

1	1	1	1	1	1	1	
			1	1	1	1	
			1	1	1	1	
		1	1	1	1	1	
			1	1	1	1	
		1	1				

(a) Binary image **B**

1	1	1
1	1	1
1	1	1

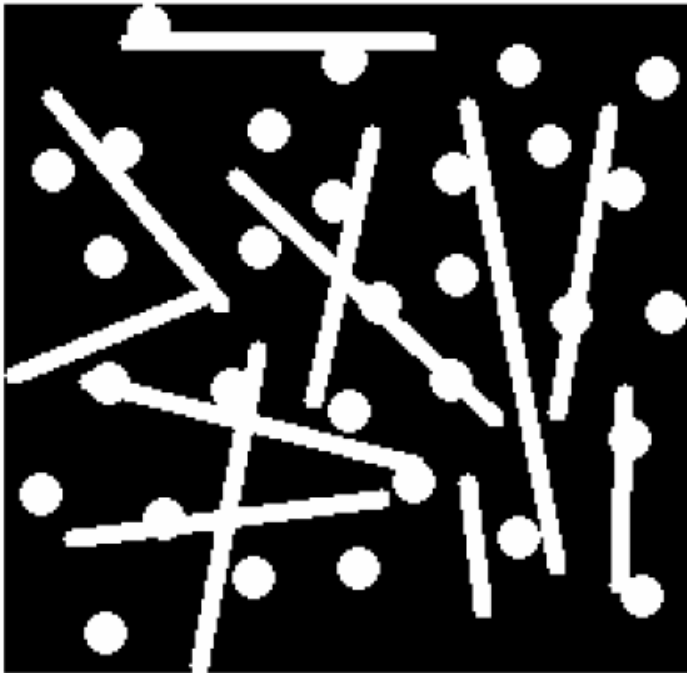
(b) Structuring element **S**

				1	1		
				1	1		
				1	1		

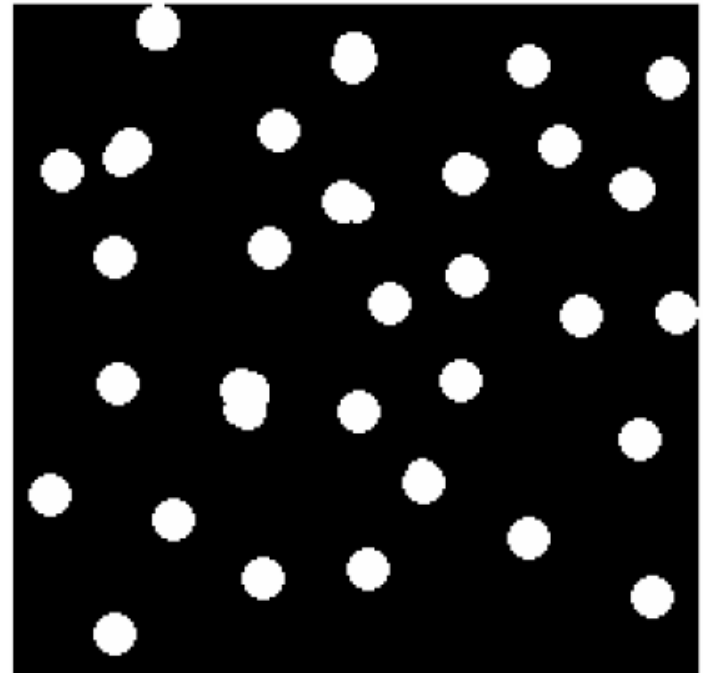
(d) Erosion $\mathbf{B} \ominus \mathbf{S}$

Opening

- Erode, then dilate
- Remove small objects, keep original shape



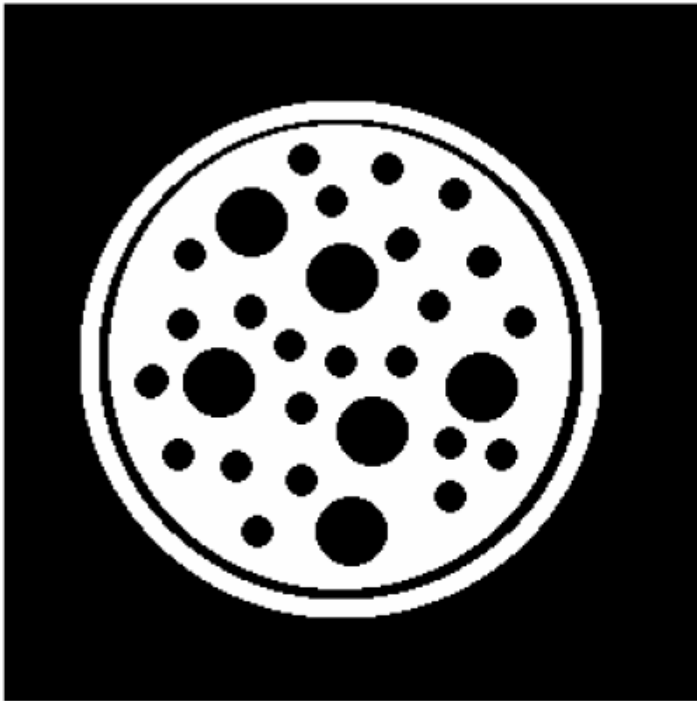
Before opening



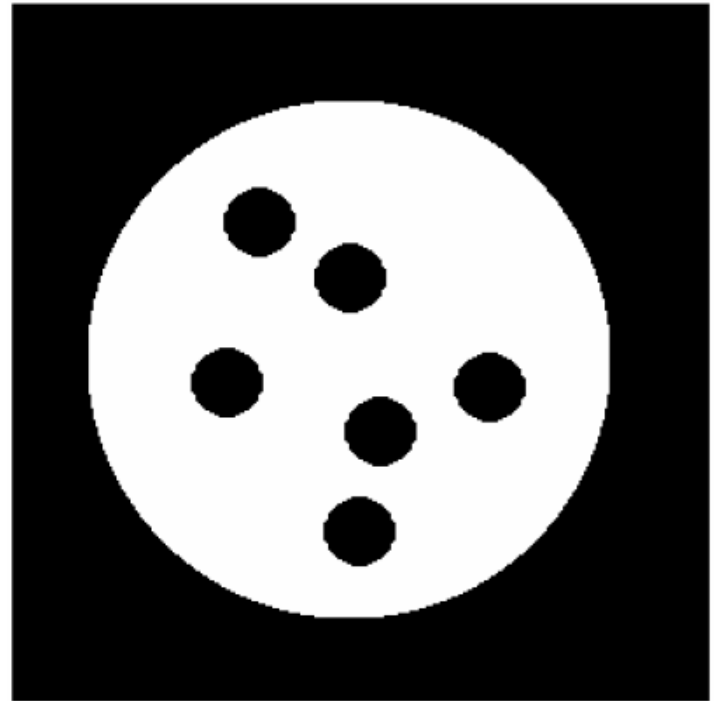
After opening

Closing

- Dilate, then erode
- Fill holes, but keep original shape



Before closing



After closing

Summary of Variants

Operation	Formula	Effect	Use Case
Dilation	Max filter	Expands bright areas	Bridging gaps, filling holes
Erosion	Min filter	Shrinks bright areas	Removing noise, separating objects
Opening	Erosion → Dilation	Removes small noise	Cleaning up white noise
Closing	Dilation → Erosion	Fills small holes	Cleaning up black noise
Gradient	Dilation - Erosion	Edge detection	Finding object boundaries
Top-Hat	Original - Opened	Enhances small bright details	Enhancing veins, fingerprint recognition
Black-Hat	Closed - Original	Enhances small dark details	Detecting dark spots, uneven lighting

Binary images: Pros and Cons

Pros:

- **Fast & Efficient** – Requires less storage and computation
- **Simple Processing** – Easy to apply operations like thresholding, morphology, and shape analysis.
- Useful for extracting object boundaries and simple geometric features.

Cons:

- Small artifacts or variations in lighting can distort object shapes.
- **Loss of Detail** – Fine textures and grayscale variations are lost, making segmentation challenging.